PHYS–467 Machine Learning for Physicists

Exercise session 11: Features, Kernels, and Neural Networks

Exercise 1: Feature Engineering

1. Open the TensorFlow playground at `https://playground.tensorflow.org/`, select the dataset with two clusters, the features x_1 and x_2 and keep zero hidden layers and linear activation. Train the model by clicking on the *play* button. Does the classifier correctly learn the boundary?

2. Now select the XOR dataset and repeat the training? Does the classifier work?

3. Keeping the same dataset, try adding and removing features to see if the performance improves? What is the minimal number of features that are needed to correctly learn the boundary? What are the features? Why do they work well?

4. Now pick the concentric circles dataset. Repeat the feature selection process from the last point and answer the same questions.

Exercise 2: Kernel Ridge Regression

1. Generate an artificial dataset with random input points X uniformly-distributed in the range $[-4, +4]$ and ground-truth function $f^*(X) = \sin(X) + 2\cos(4X) + \frac{1}{2}\sin(X+1)$. We are going to try to learn this dataset from the observations $y(X) = f^*(X) + \eta$, with $\eta \sim \mathcal{N}(0, 1)$. Plot $n = 50$ training points with their labels.

2. Try to learn this function using `sklearn.kernel_ridge.KernelRidge` with a linear 'kernel', and regularization strength $\alpha = 0.01$. Plot the predictor on the test set and compare with the ground-truth. Comment.

3. Repeat substituting the linear 'kernel' first with the RBF and then with the Laplacian kernels. How do the functions learned by these kernels differ? Why?

Exercise 3: Kernelized Logistic Regression

What about classification? Previously, in the class, we have seen that a linear model for classification is logistic regression, which solves the following optimization problem

$$\min_w \sum_{i=1}^{n} \log(1 + \exp(-y_i(X_i \cdot w))).$$

Can we use it with kernels to classify non-linearly-separable data? The answer is yes, by mapping the data in the feature space, i.e. $X_i \to \phi(X_i)$, and applying the representer theorem, i.e. $w = \sum_{j=1}^{n} \alpha_j \phi(X_j)$, we obtain

$$\min_\alpha \sum_{i=1}^{n} \log\left(1 + \exp\left(-y_i\left(\phi(X_i)\sum_{j=1}^{n}\alpha_j\phi(X_j)\right)\right)\right),$$

$$\min_\alpha \sum_{i=1}^{n} \log\left(1 + \exp\left(-y_i\left(\sum_{j=1}^{n}\alpha_j K(X_i, X_j)\right)\right)\right).$$

Therefore, it is sufficient to substitute the data matrix X with the kernel matrix $K_{ij} = K(X_i, X_j)$ in the sklearn algorithm!

1. Using `sklearn.linear_model.LogisticRegression`, fit the two-moons dataset with the RBF kernel and predict the value of the model on the test points. You can use `sklearn.metris.pairwise` to compute the kernel. Set the regularization strength $\gamma = 10$. Use the code in the notebook to plot the training points and the decision boundary.

2. Repeat with the Laplacian kernel. How do the decision boundaries learned with the two kernels differ? Why?

Exercise 4: Artificial Neural Networks with Simple Datasets

1. Select the spiral dataset and train the model as done for the previous datasets. Is there any feature combination that can be used to correctly learn the boundary?

2. Now consider the solely x_1 and x_2 features, play with the neural network architecture by progressively adding hidden layers and hidden units, try different activation functions and tune the neural network hyperparameters, i.e. the learning rate, the batch size.... Is the neural network able to learn the spiral? If yes, which is a suitable combination of network's architecture, hyperparameters and activation function that does the job?

3. Do the same for the XOR and the concentric circles datasets.